Deterministic Frontier-Scale Language Model Inference with Signed Receipts

Aishwary Singh
hhaishwary@gmail.com
OCX Protocol

April 2026

Abstract

We describe a protocol that produces byte-identical outputs from frontier-scale language model inference and binds each output to a portable, offline-verifiable signed receipt. The construction has three parts. First, an inference substrate that runs models up to seventy-two billion dense parameters and forty-seven billion mixture-of-experts active parameters on NVIDIA H100, with cross-vendor extension to AMD Instinct MI300X for the single-GPU configuration; output hashes match byte-for-byte across fresh process launches, and for the configurations we measured the AMD and NVIDIA hashes are themselves byte-identical, including over fifty-one tokens of compounding frontier-scale generation. Second, a canonical CBOR receipt schema with an Ed25519 signature over a domain-separated message, implemented in Go, Python, and Rust, with cross-language byte-identity verified end-to-end and AMD-produced receipts verifying byte-for-byte through a Rust verifier built on x86 NVIDIA hardware. Third, a probabilistic spot-check verifier that re-executes a small sample of receipts and rejects on mismatch; we prove a soundness lemma of the form $1 - (1 - f)^k$ and validate it empirically across seventy adversary-verifier configurations with seven hundred thousand Monte Carlo trials. Verification costs about eighty microseconds per receipt on a single core. Eleven thousand sequential warm-model inferences ran without a single byte-identity failure. The contribution is the construction itself: a primitive that gives issuer-independent fabrication soundness for AI inference at production cost, without a hardware-vendor dependency and without zero-knowledge proofs.

1 Introduction

A bank approves a loan. An insurer denies a claim. A hospital pre-authorises a procedure. Increasingly each of these decisions is made, in part or in whole, by a language model running on a GPU cluster owned by the institution making the decision. Six months later the customer asks why. The institution cannot answer. They can produce the output the model returned, but they cannot prove that the model they say they ran is the model that actually ran, on the inputs they say they used, with the parameters they say were in force at that time. The model output is a string in a database. The string carries no audit weight.

This is not a hypothetical. The European Union's Artificial Intelligence Act, in force from August 2024, requires traceability for high-risk AI systems [1]. The Reserve Bank of India is preparing analogous requirements for credit decisioning. The U.S. Consumer Financial Protection Bureau has signalled that "the model said so" is not a valid explanation for an adverse action. The need for an audit primitive that binds an AI inference output to a verifiable record of its production is becoming a regulatory floor.

The current answers do not solve the practical case. Zero-knowledge machine learning [2, 3] produces a cryptographic proof that a circuit was correctly evaluated, but proving a forward pass through a seventy-billion-parameter language model is far beyond what current proof systems can handle at production cost. Trusted execution environments [4, 5] allow a hardware attestation that a binary ran inside an enclave, but this trades issuer trust for hardware-vendor trust and inherits the long history of microarchitectural attacks against such enclaves. Append-only transparency logs in the Certificate Transparency tradition [7] prove that a record was published, not that it was correct. Internal audit logs prove only what the issuer chose to write down.

What is missing is a primitive that gives issuer-independent assurance of computational correctness, without a hardware vendor in the trust path and without a cryptographic proof system that does not yet scale. This paper presents that primitive. The construction has three load-bearing properties:

1. **Byte-identical output.** The same model on the same input under the same configuration produces the same output bytes across cold-started processes. We demonstrate this for dense models up to seventy-two billion parameters and mixture-of-experts models with eight experts and forty-seven billion active parameters per token, running on one and two H100 accelerators with three different parallelism configurations, and on AMD Instinct MI300X for the single-GPU configuration. For the configurations we measured, the AMD and NVIDIA hashes agree byte-for-byte, including a fifty-one-token continuation of a frontier-scale 72B model.
2. **Canonical signed receipt.** A two-hundred-byte CBOR receipt binds the model hash, input hash, output hash, parallelism configuration, and environment fingerprint under an Ed25519 signature with an OCX-specific domain separator. The same canonical bytes are produced byte-for-byte by independent Go, Python, and Rust implementations and accepted byte-for-byte by a single canonical verifier.
3. **Probabilistic soundness.** A verifier samples k receipts from a stream of N and re-executes them. The probability of catching a fabrication-rate-f issuer is $1 - C(N - fN, k)/C(N, k) \geq 1 - (1 - f)^k$. We validate this bound empirically against five adversary strategies and two verifier strategies over seventy configurations with ten thousand trials each.

The remainder of the paper specifies the protocol (§3), describes the implementation (§4), presents the empirical results (§5), proves the soundness analysis (§6), and is honest about what the protocol does not provide (§7). Every numerical result in this paper points at a specific committed file in the public repository at https://github.com/KuroKernel/ocx-protocol.

2 Related Work

Verifiable computation for machine learning. Zero-knowledge proof systems for neural network inference have advanced rapidly. EZKL [2] and similar systems based on Halo2 or Plonk can prove the evaluation of small to medium models. The Risc Zero zkVM [3] extends this to general-purpose execution traces. The technology is sound. The limitation is cost: proving a single forward pass through a seventy-billion-parameter language model under any current zk-SNARK construction is many orders of magnitude away from production economics. Our protocol gives up the unconditional soundness of zero knowledge in exchange for a cost-bounded soundness based on probabilistic re-execution. The trade is favourable in the regime where re-execution is feasible but a full proof is not.

Trusted execution environments. Intel SGX [4], AMD SEV-SNP, and NVIDIA H100 Confidential Computing [5] allow a binary to run inside an attested enclave whose state is opaque to the host. A relying party verifies the enclave's measurement and trusts the hardware vendor's signature. The disadvantages are well documented [6]: a long history of side-channel and microarchitectural attacks, vendor lock-in, and the requirement that the relying party trust the vendor's attestation root. Our protocol does not require any hardware vendor to be honest; it requires only that the deterministic-inference substrate produces the same output for the same input on the same hardware, which we demonstrate empirically.

Transparency logs. Certificate Transparency [7] and successor systems built on Trillian provide an append-only Merkle log that auditors can monitor for split-views. CT proves that a certificate was published, not that the issuing CA was honest in issuing it. The protocol described here addresses the orthogonal question: assuming receipts are published, can a verifier check that the claimed computation actually produced the claimed output? The two layers compose; in production OCX receipts can be anchored into a CT-style log to add a publication-history layer to the correctness layer described here.

Determinism in machine learning. Reproducibility has been studied at training time, where stochastic data ordering and accumulator non-determinism dominate the picture [15, 14]. Inference-time determinism has received less attention because the practical demand has been weak: most inference workloads do not require byte-identity. The PyTorch documentation describes the configuration we use; the contribution here

is to push that configuration to seventy-billion-plus parameter models across multiple parallelism strategies and to demonstrate that the resulting byte-identity is stable across thousands of warm-model iterations and across fresh process launches.

Position. OCX occupies a previously unfilled position in this matrix: issuer-independent soundness of computational correctness, with offline microsecond-scale verification, no hardware-vendor dependency, no proof-system dependency, and demonstrated operation at frontier model scale.

3 Protocol

3.1 Receipt schema

A receipt is a CBOR map with positive-integer keys. The signed core has eleven entries:

```
1: program_hash (32-byte SHA-256 of the model bytes + config)
2: input_hash (32-byte SHA-256 of the canonical input)
3: output_hash (32-byte SHA-256 of the generated tokens + decoded text)
4: gas_used (unsigned integer; tokens generated)
5: started_at (unsigned integer; UTC seconds)
6: finished_at (unsigned integer; UTC seconds)
7: issuer_id (UTF-8 text string; max 256 bytes, no control chars)
9: key_version (unsigned integer)
10: nonce (16-byte random)
11: issued_at (unsigned integer; UTC seconds)
12: float_mode (unsigned integer; reserved for future quantisation hints)
```

The wire envelope adds a 64-byte Ed25519 signature, the issuer's public key reference, and a host-info map containing the runtime fingerprint (driver version, CUDA version, PyTorch version, GPU UUID, parallelism configuration). Total receipt size is two hundred to two hundred ten bytes.

3.2 Canonical encoding

The CBOR encoding follows RFC 8949 deterministic encoding rules [8]: shortest-form integer encoding, definite-length maps and arrays, map keys sorted by their canonical byte representation, and no indefinite-length items. Floating-point numbers are not used in the signed core; all numerical fields are unsigned integers. The choice of integer keys, rather than text keys, is deliberate: it removes any ambiguity about UTF-8 normalisation and reduces the receipt by approximately fifty bytes per entry.

The canonical bytes are produced byte-identically by three independent implementations: `pkg/receipt/canonical.go` in Go, `whitepaper-tests/cross_language_roundtrip.py` in Python (using `cbor2` with `canonical=True`), and `libocx-verify/src/canonical.rs` in Rust. Cross-language byte parity is enforced by a test that fails the build on any divergence.

3.3 Signing

The signing message is

$$m = \texttt{"OCXv1|receipt|"} \parallel \texttt{canonical_cbor}(\texttt{signed_map}), \tag{1}$$

where the domain separator `"OCXv1|receipt|"` is a fourteen-byte ASCII prefix unique to the OCX receipt construction. The signature is Ed25519 [9] over m. The domain separator prevents cross-protocol signature reuse: an Ed25519 signature over OCX canonical bytes cannot be replayed against any other protocol that signs canonical CBOR with a different (or empty) prefix.

3.4 Environment binding

Byte-identity of inference depends on the runtime: NVIDIA driver version, CUDA toolkit version, PyTorch version, GPU SKU (and to some degree GPU UUID, though we have not observed a difference between two H100s in the same chassis), parallelism configuration, and the deterministic-algorithm flags described in §4. The receipt envelope binds these values explicitly in its host-info map. A verifier that re-executes for a spot-check uses the same environment values; they are part of the input contract.

4 Implementation

4.1 Deterministic inference substrate

Inference runs under PyTorch [10] 2.10 (stable Hopper) or 2.12-nightly (Blackwell) with the following configuration applied before any model load:

```
os.environ["CUBLAS_WORKSPACE_CONFIG"] = ":4096:8"
torch.use_deterministic_algorithms(True, warn_only=True)
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False
torch.backends.cuda.matmul.allow_tf32 = False
torch.backends.cudnn.allow_tf32 = False
torch.manual_seed(42); torch.cuda.manual_seed_all(42)
```

Models are loaded with `torch_dtype=torch.bfloat16` and `attn_implementation="eager"`. We do not use Flash Attention [13]: its CUDA implementation is not bit-deterministic across batches in the configurations we tested. Greedy decoding is used throughout (`do_sample=False`, `temperature=0`, `num_beams=1`).

For tensor-parallel inference across two GPUs we use the HuggingFace Transformers [11] `tp_plan="auto"` option, which dispatches NCCL [12] ring all-reduce over the NV18 NVLink fabric between the two H100 accelerators. The reduction order within each all-reduce is fixed by the ring topology, which in turn fixes the bit pattern of the reduced tensor across runs. For pipeline-parallel and CPU-offloaded configurations we use `device_map="auto"`.

4.2 Verification stack

The canonical verifier is a Rust crate, `libocx-verify`, exposing a C ABI through cbindgen. The single entry point is

```
bool ocx_verify_receipt_detailed(
    const uint8_t* cbor_data, size_t cbor_len,
    const uint8_t* public_key, int* error_code_out);
```

It parses the CBOR, validates schema constraints, reconstructs the signing message, and verifies the Ed25519 signature. On error it returns one of seven codes covering invalid CBOR, non-canonical CBOR, missing field, invalid field value, invalid signature, hash mismatch, and invalid timestamp. The library is linked into the Python and Go reference clients via `ctypes` and `cgo` respectively; both clients call the same compiled `liblibocx_verify.so` so there is exactly one canonical verifier in the system.

4.3 Deployment

The reference issuer exposes an OpenAI-compatible HTTP endpoint at `/v1/chat/completions`. Each response carries the receipt in two HTTP headers: `X-OCX-Receipt-B64` (base64-encoded receipt CBOR) and `X-OCX-PublicKey-Hex`. A client receives the regular OpenAI-shaped JSON response, plus the receipt headers. Verification is a single FFI call against `libocx-verify`; nothing about the API surface differs from a non-verified OpenAI-compatible endpoint.

5 Empirical Results

All measurements are reproducible from the public repository. The numerical claims below cite specific committed files; each receipt JSON contains the inputs, the parallelism configuration, the elapsed time, and the verification result. The full test plan and aggregated results are in `whitepaper-tests/TEST_PLAN.md` and `whitepaper-tests/TEST_RESULTS.md`.

5.1 Cross-language byte parity

A receipt produced by the Go canonicaliser, signed by a Python signer, and verified by the Rust FFI: eight assertions covering canonical-byte equality, signature byte-identity across signer implementations, verification success, signature tamper-detection, payload tamper-detection, and wrong-key rejection. **8 of 8 pass.**

Test category	Passing
Go core protocol (canonicalisation, schema, signature)	147 / 147
Rust `libocx-verify` (parsing, FFI, validation)	79 / 79
Cross-language Go ↔ Python ↔ Rust round-trip	8 / 8
Bench (10 000 distinct receipts verified)	10 000 / 10 000

Table 1: Layer-1 and Layer-2 protocol tests.

5.2 Verification performance

Ten thousand distinct receipts, each verified through the Rust FFI from a single Python thread on commodity x86_64 hardware (Intel i7-class, no GPU): median 79.4 μs, p99 114.6 μs, sustained throughput 12 392 receipts/sec on a single core. The receipt size on the wire is 200–210 bytes. Verification is dominated by the Ed25519 verification call ($\approx 60\,\mu$s); CBOR parsing and schema validation account for the remainder.

5.3 Byte-identical inference at frontier scale

We demonstrate byte-identical output across fresh `torchrun` launches for three models and three parallelism configurations. Each row is a group of fresh, cold-started Python processes. Within each group, every receipt has identical `output_hash` and identical `logits_hash`.

The mixture-of-experts result (Mixtral rows) is the load-bearing one. The standard objection to deterministic inference is that mixture-of-experts routing is stochastic. Mixtral's router selects two of eight experts per token via arg max over routing logits. The arg max is deterministic when the routing logits are deterministic, and the logits are deterministic when the matrix multiplications producing them are bit-stable. Across three fresh `torchrun` launches we observed identical `logits_hash` as well as identical `output_hash`, which establishes that the router itself is byte-stable, not merely that the final output happened to match.

The pipeline-parallel and tensor-parallel rows produce *different* hashes from each other (the reduction order in NCCL ring all-reduce differs from the single-stream order on one GPU), but each is internally byte-identical across runs. We treat this as part of the input contract: a verifier re-executing for a spot-check uses the same parallelism configuration and gets the same hash.

5.4 Long-run warm-model stability

Skeptics of deterministic inference often suggest that thermal drift, accumulated allocator state, or KV-cache non-determinism will break byte-identity over hours of continuous load. We tested this on two models:

- **Mixtral 8x7B-Instruct, 1 000 sequential inferences, 18 minutes wall.** Ten distinct prompt classes, each prompt run 100 times. **0 / 1 000 byte-identity failures.** GPU memory drift over the run: 2 MiB. Thermal range: 33 °C → 44 °C.
- **Qwen 2.5-0.5B-Instruct, 10 000 sequential inferences, 48 minutes wall.** Same protocol. **0 / 10 000 byte-identity failures.** GPU memory drift: 0 MiB. Thermal range: 30 °C → 32 °C.

Model	Parallelism	Generation	Runs	Result
Qwen 2.5-72B-Instruct	1×H100 + CPU offload, bf16	128 tokens	3	byte-identical
Qwen 2.5-72B-Instruct	2×H100 pipeline (`device_map=auto`), bf16	128 tokens	1	matches CPU-offload
Qwen 2.5-72B-Instruct	2×H100 tensor (`tp_plan=auto`), bf16	32 tokens	3	byte-identical
Qwen 2.5-72B-Instruct	2×H100 tensor, bf16	128 tokens	3	byte-identical
Llama 3.1-70B-Instruct	2×H100 tensor, bf16	32 tokens	3	byte-identical
Llama 3.1-70B-Instruct	2×H100 tensor, bf16	128 tokens	3	byte-identical
Mixtral 8x7B-Instruct (MoE)	2×H100 tensor, bf16	32 tokens	3	byte-identical
Mixtral 8x7B-Instruct (MoE)	2×H100 tensor, bf16	128 tokens	3	byte-identical
Cross-vendor: AMD Instinct MI300X (CDNA3 gfx942), ROCm 6.1, PyTorch 2.4.1+rocm6.1				
Qwen 2.5-0.5B-Instruct	1×MI300X, bf16, eager attn	32 tokens	3	byte-identical *and matches H100*
Qwen 2.5-72B-Instruct	1×MI300X, bf16, eager attn	5 tokens	3	byte-identical *and matches H100*
Qwen 2.5-72B-Instruct	1×MI300X, bf16, eager attn	51 tokens	3	byte-identical *and matches H100*

Table 2: Byte-identical inference across fresh `torchrun` launches. The first eight rows: 22 launches across 8 model×configuration groups on NVIDIA Hopper, every group internally byte-identical. The last three rows: 9 launches across 3 model×configuration groups on AMD CDNA3, every group internally byte-identical *and* matching the corresponding NVIDIA H100 single-GPU CPU-offload baseline byte-for-byte across vendors.

Combined: **11 000 sequential warm-model inferences, 0 byte-identity failures, 66 minutes of continuous load, 2 MiB peak memory drift.** Per-iteration timing was stable to within $\pm 5\,\%$; we observed no monotonic latency degradation that would indicate runaway state accumulation.

5.5 Cross-session and cross-vendor reproducibility

The headline determinism claim concerns reproducibility across fresh process launches in the same session. A stronger claim is reproducibility across separately provisioned hardware, including across hardware vendors.

Cross-session, same vendor. The Qwen 2.5-0.5B-Instruct "capital of France" prompt produced `output_hash` `0fbfb8ec...` on NVIDIA RTX 5060 (Blackwell sm_120, PyTorch 2.12-nightly) on day one, and on NVIDIA H100 SXM (Hopper sm_90, PyTorch 2.10 stable) three weeks later, on a different physical machine in a different country. The same hash, across architectures within NVIDIA's lineage.

Cross-vendor. We extended the test to AMD Instinct MI300X (CDNA3 gfx942), running PyTorch 2.4.1+rocm6.1 against ROCm 6.1, single-GPU bf16 with eager attention. The configuration is the smallest deviation from the NVIDIA H100 single-GPU CPU-offload baseline: same model bytes, same prompt bytes, same dtype, same sampling parameters. Three fresh launches per (model, prompt) combination on MI300X produced the following `output_hash` values, each compared to the corresponding H100 baseline:

Model	Tokens	Runs	MI300X hash (3/3 identical)	H100 hash (3/3 identical)
Qwen 2.5-0.5B-Instruct	32	3	`0fbfb8ec...`	`0fbfb8ec...` *(match)*
Qwen 2.5-72B-Instruct	5	3	`fe27f5da...`	`fe27f5da...` *(match)*
Qwen 2.5-72B-Instruct	51	3	`e59fca7d...`	`e59fca7d...` *(match)*

Table 3: Cross-vendor byte-identity. Same prompt, same dtype, same eager attention, same greedy sampling. AMD MI300X (CDNA3) and NVIDIA H100 (Hopper) produce the same `output_hash` byte-for-byte across vendors.

This is a stronger result than we expected at the time of writing the protocol description (§3). Our prior view, and the conventional one, was that floating-point reduction order would differ between vendors and produce different output hashes. For these configurations and these models, the reduction trees implemented by AMD's hipBLAS on CDNA3 and NVIDIA's cuBLAS on Hopper are functionally identical at bf16 precision, and the resulting output hash is portable across vendors. The 51-token result is the load-bearing one: it composes 51 forward passes through 80 transformer layers, each containing many matrix multiplications and reductions, and the cumulative bit pattern still matches.

We do not claim this generalises. Tensor-parallel configurations using NCCL ring all-reduce on NVIDIA versus RCCL fabric all-reduce on AMD will likely produce different reduction orders, and we have not yet measured them. The small-model cross-architecture result on Blackwell vs Hopper above similarly cannot be assumed to generalise to all architectures: ARM64 vs x86_64 CPU floating-point libraries diverge at LLM scale (we measured this in earlier work). The receipt's environment binding remains the protocol-level mechanism that handles the general case where outputs differ; the cross-vendor empirical result is a useful surprise, not a guarantee.

The MI300X receipts are committed under `examples/gpu-verifier/results/mi300x/` and verify byte-for-byte through the canonical Rust verifier built on x86_64 NVIDIA hardware. The protocol's receipt layer is vendor-portable by construction; the empirical evidence here is that, for these configurations, the protocol's *computation* layer is too.

6 Soundness

The protocol gives a verifier the ability to re-execute any receipt and check the result. Soundness is the question of how much re-execution is enough.

6.1 Threat model

Setting. An issuer I produces a stream of N signed receipts. A verifier V wants assurance that for each i, the receipt's `output_hash` equals the byte-identical hash that an honest re-execution of `model`(`input`$_i$) would produce.

Adversary capabilities. The adversary can sign anything with their valid Ed25519 key (signatures alone do not bind correctness), can choose per receipt whether to compute honestly or fabricate the `output_hash`, can observe public attributes of each receipt (which input class, whether labelled high-stakes) before committing, and can substitute any 256-bit value for the `output_hash` including a value drawn from a different honest computation (a replay).

Adversary limitations. The adversary cannot forge another issuer's signature (Ed25519 unforgeability), cannot break SHA-256 collision resistance (so a fabricated hash will not coincide with the honest hash for any input the verifier knows about), cannot influence the verifier's challenge coin (in production this is enforced by deriving the challenge schedule from a public beacon committed *after* receipts are issued), and cannot tamper with the verifier's local re-execution.

Trust assumptions. Soundness depends on Ed25519, SHA-256, canonical CBOR, and the verifier's own deterministic re-execution capability (proven in §5). It does *not* depend on hardware vendor honesty, on the existence of a scalable zero-knowledge proof system, or on a trusted third party.

Security game. (1) The adversary publishes a strategy and N. (2) The adversary issues N receipts. (3) The verifier samples k indices from a public coin, re-executes those k inferences, and emits ACCEPT iff every sample matches its re-execution. (4) The adversary wins if at least one fabricated receipt is in the stream and the verifier emits ACCEPT.

6.2 Spot-check soundness

Lemma 1 (Hypergeometric soundness, uniform sampling)**.** *Let N be the number of issued receipts, $L \le N$ the number whose* `output_hash` *differs from the honest re-execution hash, and let the verifier sample k distinct indices uniformly at random from $\{1, \ldots, N\}$, independently of the adversary's choices. Then*

$$P[\textit{verifier rejects}] \;=\; 1 - \frac{\binom{N-L}{k}}{\binom{N}{k}}.$$

Proof. By the determinism guarantee of §5, an honest re-execution of input i matches receipt i's `output_hash` if and only if receipt i is honest. The verifier rejects exactly when at least one sampled index is a fabricated receipt. The number of fabricated receipts in the size-k uniform sample is hypergeometric with parameters (N, L, k). The complement of "no fabricated receipt sampled" is the ratio in the lemma. □

Corollary 1 (Independent-sample bound)**.** *For fabrication rate $f := L/N$,*

$$P[\textit{verifier rejects}] \;\ge\; 1 - (1-f)^k.$$

This bound is tight for $L \ll N$ and is the form usually quoted.

Corollary 2 (Rate–coverage trade-off)**.** *To bound the adversary's success probability below ε with sample size k, the issuer can fabricate at most $L_{\max} \approx N\big(1 - (1-\varepsilon)^{1/k}\big) \approx N\varepsilon/k$ receipts. For $N = 10^4$, $\varepsilon = 0.01$, $k = 100$: $L_{\max} \approx 1.01$ receipt per stream.*

6.3 Risk-weighted sampling

A verifier that knows which inputs the adversary most wants to lie about (high-value loan decisions, high-stakes medical pre-authorisations, large procurement orders) can oversample those receipts. Using the Efraimidis–Spirakis [16] weighted reservoir scheme, a high-stakes receipt with weight w relative to a normal receipt's weight 1 enters the sample with probability proportional to w at each draw. Against a `SelectiveLiar` adversary that fabricates only on the high-stakes subset (5 % of the stream), the empirical catch probability with $k = 1$ rises from 0.05 (uniform) to 0.51 (risk-weighted with $w = 20$): a 9.76× improvement at the same compute budget.

6.4 Replay irrelevance

Lemma 2 (Replay irrelevance)**.** *Spot-check verification is invariant under any adversary strategy that produces a 256-bit value not equal to* `hash`(`model`(`input`$_i$)) *for input i. The adversary's win probability depends only on (L, N, k) and the verifier's sampling strategy, not on the structure of the fabricated hash.*

Proof. Re-execution computes the canonical hash for input i and tests equality against the receipt's claim. Inequality is the unique catch event. The provenance of an inequality-producing 256-bit value (random, replayed from another honest receipt, structurally crafted) does not affect the test. □

This lemma rules out the natural attack of substituting a hash that has been signed elsewhere: such a hash is in some global set of "hashes the verifier has seen", but it is not the hash for *this* input, and re-execution catches it identically to a randomly fabricated hash.

6.5 Empirical Monte Carlo validation

We implemented five adversary strategies (`HonestProver`, `PureLiar`, `BernoulliLiar`(f) for $f \in \{0.001, 0.01, 0.10\}$, `SelectiveLiar`, `ReplayAttacker`) and two verifier strategies (uniform sampling, risk-weighted sampling with $w = 20$ on a 5 % high-stakes subset). For each (adversary, verifier, k) cell with $k \in \{1, 5, 25, 100, 500\}$, we ran $M = 10\,000$ trials. The verifier's "re-execution" is modelled by a lookup against a 27-entry challenge bank built from the receipts committed in `examples/gpu-verifier/results/h100/` and the long-run JSONL summaries; each entry's hash is itself proven byte-identical across multiple fresh `torchrun` launches in §5, so a lookup is exactly what an honest re-execution would produce.

Adversary	k	$\hat{f}$	$P_{\text{empirical}}$	$P_{\text{theoretical}}$
`HonestProver`	any	0.0000	0.0000	0.0000
`PureLiar`	1	1.0000	1.0000	1.0000
`BernoulliLiar(f=0.001)`	100	0.0010	0.0973	0.0957
`BernoulliLiar(f=0.01)`	500	0.0100	0.9924	0.9942
`BernoulliLiar(f=0.10)`	25	0.1000	0.9265	0.9284
`SelectiveLiar` (uniform sampling)	1	0.0500	0.0520	0.0500
`SelectiveLiar` (risk-weighted)	1	0.0500	**0.5076**	—
`ReplayAttacker(f=0.10)`	25	0.1000	0.9274	0.9284

Table 4: Selected Monte Carlo cells. Full data: `adversarial_soundness.jsonl`. Empirical catch rate matches theoretical hypergeometric prediction within Monte Carlo error in every cell. The risk-weighted row at $k = 1$ is the 9.76× improvement over uniform sampling against a stake-targeting adversary.

Total: 70 cells, 700 000 trials, 87 seconds of single-core wall time.

Result. Across all 70 cells, $|P_{\text{empirical}} - P_{\text{theoretical}}| \leq 5\sigma$ where $\sigma = \sqrt{p(1-p)/M}$ (per-cell false-positive rate $< 6 \times 10^{-7}$). Selected representative cells:

`HonestProver` is never falsely accused (zero false-positives across 50 000 trials). `ReplayAttacker` matches `BernoulliLiar` cell-for-cell, confirming Lemma 2 empirically. The risk-weighted catch rate against `SelectiveLiar` is dramatically higher than uniform at small k (where uniform sampling has not yet saturated).

6.6 Comparison with alternatives

Approach	Soundness primitive	Trust requirement
OCX (this work)	$1 - (1-f)^k$ via re-execution	determinism + signature + verifier compute
zkML (EZKL, Risc Zero)	SNARK over circuit evaluation	cryptographic + bounded compute (does not yet scale)
TEE (SGX, H100 CC)	hardware attestation	hardware vendor + microcode integrity
Audit logs	issuer's own log	issuer honesty (no soundness against lying issuer)
Certificate Transparency	append-only Merkle log	issuer can still publish bogus records

Table 5: Soundness comparison. OCX is the first construction in this matrix that simultaneously catches a lying issuer with bounded sampling, requires no hardware-vendor honesty, and verifies in microseconds offline.

7 Limitations

The protocol does not provide zero knowledge: the verifier sees the input and output of any sampled inference. For workloads where the input is sensitive (medical records, private financial data), OCX would compose with a separate confidentiality layer; we do not address that here.

The protocol does not protect against hardware-vendor compromise of the determinism substrate. If NVIDIA shipped a driver that silently produced different outputs on alternate Tuesdays, no software-level audit would detect it. We mitigate this only weakly, by binding driver and CUDA versions in the receipt envelope and by relying on multi-vendor verification in the future as the protocol becomes cross-vendor.

The protocol's spot-check soundness collapses if the adversary can predict or influence the verifier's challenge coin. In production this requires deriving the challenge schedule from a public beacon (block hash, drand, witness signature) committed after receipts are issued. The reference implementation does not yet enforce this; it is a Layer 3 (witness consensus) extension.

Cross-architecture and cross-vendor inference *can* produce different output hashes for the same weights and inputs, by the physics of floating-point reduction order. The receipt's environment binding makes the configuration explicit, so a verifier knows which substrate to re-execute on. §5 documents one positive empirical surprise: at single-GPU bf16 with eager attention, AMD MI300X (CDNA3) and NVIDIA H100 (Hopper) produce byte-identical output hashes for Qwen 2.5-72B, including over 51 tokens of compounding

reductions. We have not yet measured tensor-parallel configurations across vendors (NCCL ring vs RCCL fabric is the most likely point of divergence), nor Intel Gaudi or Google TPU.

We have not measured determinism in vLLM or other production-throughput inference stacks. PagedAttention is documented as non-deterministic; whether it can be made byte-deterministic with reasonable engineering effort remains open.

OCX cannot make a closed-source model auditable. The protocol assumes the verifier can re-execute the model. For OpenAI, Anthropic, and Google models this would require the model provider to participate in the protocol. The construction here is therefore most immediately applicable to open-weight deployments (Llama, Qwen, Mixtral, DeepSeek, and successors).

8 Conclusion

Two questions point forward. First, when do output hashes become vendor-neutral? The cross-vendor byte-identity we observe between NVIDIA Hopper and AMD CDNA3 for single-GPU bf16 with eager attention is a positive surprise (§5); it implies that for some non-trivial subset of frontier-scale workloads, the reduction trees implemented by hipBLAS and cuBLAS happen to coincide. The open question is what conditions are sufficient (tensor-parallel almost certainly diverges; what about Flash Attention v3 once it is bit-deterministic; what about Intel Gaudi or Google TPU?), and whether a deliberate construction over a fixed-point or arbitrary-precision intermediate could unify receipts across all substrates at a controlled accuracy cost. Second, can the challenge coin be removed from the trust path entirely? A witness-consensus layer in which independent observers each contribute entropy to the challenge schedule, and the verifier samples against the joint coin, would close the adaptive-challenge gap without requiring a global beacon.

Neither extension changes the construction in this paper. The primitive stands on its own: deterministic frontier-scale language model inference, signed canonical receipts that verify offline in microseconds, soundness against a lying issuer with provable bounds. We hope it is useful.

Reproducibility. Every measurement in this paper points at a specific committed file at https://github.com/KuroKernel/ocx-protocol. Receipts: `examples/gpu-verifier/results/h100/`. Test plan: `whitepaper-tests/TEST_PLAN.md`. Monte Carlo data: `examples/gpu-verifier/results/h100/adversarial_soundness.j`
A reviewer can verify any claim by running the listed commands on the listed commit.

Acknowledgments. The protocol benefited from cloud GPU credits at JarvisLabs.ai. The authors thank the open-weight model providers (Alibaba Qwen team, Meta Llama team, Mistral) whose models made the determinism evidence possible.

References

[1] European Parliament and Council. *Regulation (EU) 2024/1689 (Artificial Intelligence Act)*. Article 14 (Human Oversight) and Annex IV (Technical Documentation). 13 June 2024.

[2] EZKL Project. *EZKL: Zero-knowledge proofs for machine learning models*. https://github.com/zkonduit/ezkl, 2024.

[3] Risc Zero Inc. *The Risc Zero zkVM: a general-purpose zero-knowledge virtual machine*. Technical report, 2023.

[4] V. Costan and S. Devadas. *Intel SGX Explained*. IACR Cryptology ePrint Archive, Report 2016/086, 2016.

[5] NVIDIA Corporation. *NVIDIA H100 Tensor Core GPU Architecture*. Whitepaper, sections on Confidential Computing, 2022.

[6] J. Van Bulck et al. *Foreshadow: Extracting the keys to the Intel SGX kingdom with transient out-of-order execution*. USENIX Security, 2018.

[7] B. Laurie, A. Langley, and E. Kasper. *Certificate Transparency*. RFC 6962, IETF, 2013.

[8] C. Bormann and P. Hoffman. *Concise Binary Object Representation (CBOR)*. RFC 8949, IETF, 2020.

[9] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, and B.-Y. Yang. *High-speed high-security signatures*. Journal of Cryptographic Engineering, 2(2):77–89, 2012.

[10] A. Paszke et al. *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. NeurIPS, 2019.

[11] T. Wolf et al. *Transformers: State-of-the-Art Natural Language Processing*. Proceedings of EMNLP: System Demonstrations, 2020.

[12] S. Jeaugey. *NCCL 2.0: A Library for Optimized Collective Communications*. NVIDIA GTC, 2017.

[13] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. NeurIPS, 2022.

[14] PyTorch Project. *Reproducibility*. Documentation. https://pytorch.org/docs/stable/notes/randomness.html.

[15] P. Nagarajan, G. Warnell, and P. Stone. *Deterministic Implementations for Reproducibility in Deep Reinforcement Learning*. AAAI Workshop, 2019.

[16] P. S. Efraimidis and P. G. Spirakis. *Weighted random sampling with a reservoir*. Information Processing Letters, 97(5):181–185, 2006.

[17] S. Nakamoto. *Bitcoin: A peer-to-peer electronic cash system*. 2008.

[18] A. Vaswani et al. *Attention Is All You Need*. NeurIPS, 2017.

[19] A. Dubey et al. *The Llama 3 Herd of Models*. Meta AI, 2024.

[20] A. Q. Jiang et al. *Mixtral of Experts*. Mistral AI, 2024.

[21] A. Yang et al. *Qwen2.5 Technical Report*. Alibaba Cloud, 2024.